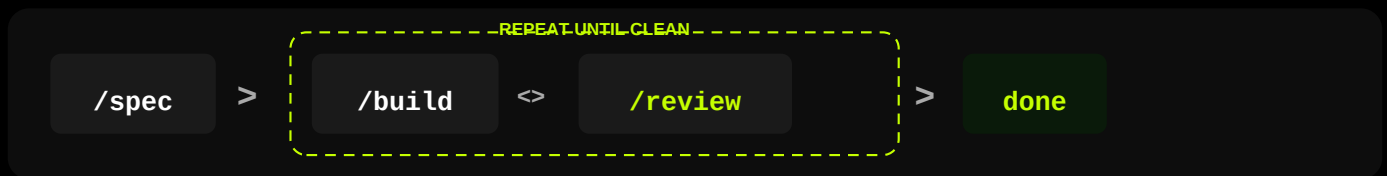


CLAUDE CODE . LOOPS

LOOPS EN CLAUDE CODE

Construye un sistema que se corrige solo.

Tres skills que trabajan en ciclo: tu idea entra como concepto y sale como build verificado. Sin supervision. Sin prompts repetidos.



CLAUDE CODE

PRODUCTIVIDAD

AUTOMATIZACION

GRATIS

UN PROMPT ES...

Un intento.

Le pides algo a Claude, recibes lo que sea, y eso es todo. Un solo disparo, sin verificación.

UN LOOP ES...

Un sistema.

Claude ejecuta, evalúa su resultado, corrige lo que falla, y repite hasta que este bien.

CLAVE

Separar la ejecución de la evaluación es lo que hace la diferencia. Claude no puede editar bien lo que acaba de escribir. Pero si primero califica y luego reescribe, cada ciclo se acerca más al resultado correcto.

Como funciona el ciclo:

/spec Entiende que quieres construir y escribe una spec detallada.

/build Lee la spec y construye exactamente lo que dice, sin adivinar.

/review Compara el build contra la spec. Si falla, lo devuelve a /build.

done El loop para cuando el review pasa limpio. Sin intervención tuya.

02

LAS 3 SKILLS

Cada una hace una sola cosa — las tres juntas hacen todo

01

/spec

Convierte tu idea en un plan

Te hace preguntas una por una hasta entender exactamente que quieres construir: el objetivo, los requisitos, las restricciones y que significa que este terminado. Escribe una spec detallada en specs/<nombre>.md. No empieza a construir hasta que lo indiques.

02

/build

Construye directo desde la spec

Lee specs/<nombre>.md y construye exactamente lo que dice. No agrega features, no refactoriza código sin relación, no inventa requisitos. Cuando termina, lista que requisitos cubrió para que /review los pueda verificar uno por uno.

03

/review

Califica el build, devuelve los fallos

Compara el build actual contra specs/<nombre>.md requisito por requisito. Lista cada gap, bug o pieza faltante, nombrando exactamente que punto de la spec falla. Si algo no está completo, escribe los fixes específicos y se los devuelve a /build. Solo aprueba cuando TODO está cumplido.

Copia cada prompt y pegalo en Claude Code — una vez cada uno

Convierte tu idea en un plan

Copia este prompt completo y pegalo directamente en Claude Code.

Claude Code creara el skill /spec automaticamente. Solo ejecutas esto una vez.

PEGA ESTO EN CLAUDE CODE

Create a Claude Code skill called "spec".

When I run /spec, interview me about the feature or app I want to build. Ask one focused question at a time until you fully understand the goal, the must-have requirements, the constraints, and what "done" looks like. Do not start building.

When you have enough, write a clear, detailed spec and save it to specs/<name>.md. The spec must include: the objective, the exact requirements, the edge cases to handle, and a concrete definition of done that someone could check the build against.

Que hace este skill:

Te entrevista: pregunta una cosa a la vez hasta entender exactamente que quieres.

Escribe la spec: crea specs/<nombre>.md con objetivo, requisitos y definicion de done.

No construye: su unico trabajo es entender y documentar. Cero codigo en este paso.

Lee la spec y construye exactamente lo que dice. Sin features extras, sin scope creep, sin adivinar.

PEGA ESTO EN CLAUDE CODE

Create a Claude Code skill called "build".

When I run /build, read the spec in specs/<name>.md and build exactly what it describes. Do not add features, refactor unrelated code, or invent requirements that aren't in the spec. When you finish, list which spec requirements you covered so the review step can check them.

Compara el build contra la spec linea a linea. Si algo falla, escribe los fixes y se los devuelve a /build.

PEGA ESTO EN CLAUDE CODE

Create a Claude Code skill called "review".

When I run /review, compare the current build against specs/<name>.md. Go requirement by requirement and list every gap, bug, or missing piece, naming the exact spec item each one fails. If anything fails, write the specific fixes needed and hand them back so /build can address them. Only pass the build when every requirement in the spec is fully met.

EJECUTAR EL LOOP

Con las 3 skills instaladas, esto es todo lo que necesitas hacer

PASO 1 -- Definir la idea con /spec

Corre /spec una sola vez y responde sus preguntas.

Claude Code te hara preguntas sobre tu idea: objetivo, requisitos, restricciones y que significa que este terminado. Responde con detalle. Al final, /spec guarda todo en specs/<nombre>.md. Ese archivo es el contrato del loop.

PASO 2 -- Iniciar el loop automatico

Con la spec lista, pega este comando y deja que Claude lo resuelva solo:

```
Loop /build and /review: build from the spec, review the build against the spec, fix whatever fails, then repeat until the review passes clean. Keep going on your own until it passes.
```

Claude va a: (1) ejecutar /build para construir segun la spec, (2) ejecutar /review para comparar el resultado, (3) si algo falla, aplicar los fixes y volver a /build, (4) repetir hasta que el review pase limpio. Todo sin que intervengas.

CONSEJO -- Modo alto esfuerzo

Activa el modo 'high effort' en Claude Code antes de iniciar el loop. Permite que el modelo trabaje mas tiempo sin interrupciones — ideal para loops que necesitan varios ciclos de /build + /review.

El flujo completo en 2 pasos:

/spec -> defines exactamente que construir

Loop /build + /review -> Claude construye, evalua y corrige solo

Modo facil -- Un loop ya construido

Instala el skill plan-optimizer con un solo comando de terminal. Recibe cualquier plan, lo califica con una rubrica propia, lo critica y lo reescribe hasta no poder mejorar mas. Ideal para documentos, estrategias o cualquier contenido.

EJECUTA ESTO EN TU TERMINAL

```
curl -fsSL https://seangeng.com/plan-optimizer-skill.zip -o /tmp/plan-optimizer-skill.zip && unzip -o /tmp/plan-optimizer-skill.zip -d ~/.claude/skills/ && rm /tmp/plan-optimizer-skill.zip
```

Skill creado por @goodalexander y @seangeng. Se instala directamente en ~/.claude/skills/

Sin instalacion -- El loop de un parrafo

Pega esto DESPUES de cualquier tarea que le des a Claude. Funciona para escritura, planes, codigo, emails — cualquier cosa con una barra de calidad. No necesitas instalar nada, no necesitas Claude Code.

AGREGA ESTO AL FINAL DE TU PROMPT

```
After you finish, score your own work 0-100 against a clear rubric you write first. List the weakest parts and why they cost points. Rewrite to fix the top weaknesses while keeping what scored well. Repeat until the score stops improving by a real margin, then give me the best version and the score trajectory.
```

Funciona perfecto para:

- | | |
|------------------------------------|-------------------------------|
| -> Emails y propuestas comerciales | -> Scripts de video o podcast |
| -> Estrategias de contenido | -> Copies para redes sociales |
| -> Planes y reportes de clientes | -> Presentaciones de negocio |

01 Landing Page

- /spec** Secciones, CTAs y mensajes clave
- /build** Genera el HTML con todas las secciones
- /review** Verifica cada CTA y elemento requerido

02 Estrategia de Contenido

- /spec** Objetivos, audiencia y canales definidos
- /build** Plan de 30 dias con temas y formatos
- /review** Evalua coherencia y cobertura de temas

03 Script de Video

- /spec** Tono, duracion y objetivo del video
- /build** Guion completo con hook y CTA
- /review** Evalua fluidez, gancho y llamada a accion

04 Secuencia de Emails

- /spec** Audiencia, funnel y cantidad de emails
- /build** Secuencia completa con subject lines
- /review** Verifica flujo narrativo y conversion

05 Propuesta de Servicios

- /spec** Problema del cliente y servicios incluidos
- /build** Propuesta con pricing y casos de exito
- /review** Verifica que cubre todos los puntos del brief

06 Flujo de Automatizacion

- /spec** Triggers, condiciones y mensajes del flujo
- /build** Construye el flujo en ManyChat o similar
- /review** Verifica triggers, respuestas y condiciones

Deja de supervisar prompts.

Construye el sistema que prompta por ti.

Sigue a @prompteafacil para mas sistemas de IA que trabajan solos.

@prompteafacil